EXPRESSION TREE VISUALIZER

Project Report

Course Data Structures
Author Tasmai Keni
Institution DY Patil Deemed To Be University, Navi Mumbai
Year 2026

Abstract

The Expression Tree Visualizer is an interactive desktop application that demonstrates how infix expressions are tokenized, validated, converted, represented as binary trees, traversed, and — when numeric — evaluated. The system accepts expressions containing numeric literals and symbolic variables, converts them into postfix and prefix notation, constructs a binary expression tree, and presents each stage through step-by-step visualization.

The project combines a stack abstract data type for notation conversion, a binary tree for hierarchical structure, and recursive depth-first traversals for notation generation and node visitation. The graphical interface, built with PySide6, allows learners to observe algorithmic state changes, inspect traversals, and study the relationship between expression syntax and tree structure.

1 Problem Statement

Expression processing is one of the clearest examples of multiple data structures working together in a single pipeline. A learner must understand how tokens are produced, how precedence and parentheses govern operator ordering, how postfix notation can be mapped into a tree, and how traversals reveal different notational views of the same expression.

The challenge is to present this pipeline in a form that is both rigorous and accessible. Expressions such as ((3+4)/(9-2)), (A+B), and ((x+y)/(z-w)) should all be interpreted structurally, producing correct prefix and postfix forms and a correct binary tree. Numeric expressions can then proceed to evaluation, while symbolic expressions remain useful for visualization and traversal study.

The application therefore aims to provide a single environment in which users can:

1. Enter a valid infix expression composed of operators, parentheses, numeric literals, and symbolic variables.
2. Observe infix-to-postfix conversion through the Shunting-Yard algorithm.
3. Build the corresponding expression tree from postfix notation.
4. Explore inorder, preorder, and postorder traversals.
5. Evaluate the tree whenever every operand is numeric.

2 Objectives

#	Objective	Category
1	Validate infix expressions with clear syntax feedback	Usability

#	Objective	Category
2	Convert infix expressions to postfix notation	Algorithm
3	Convert infix expressions to prefix notation	Algorithm
4	Construct a binary expression tree from postfix tokens	Data Structure
5	Visualize stack operations and tree construction	Visualization
6	Support inorder, preorder, and postorder traversals	Algorithm
7	Evaluate numeric expression trees through postorder recursion	Algorithm
8	Present the workflow through an interactive GUI	Engineering

3 Expression Trees and Related Theory

3.1 Stack Data Structure

A stack is a Last-In, First-Out (LIFO) structure. It is essential in infix conversion because operators must sometimes wait until higher-precedence operations or parentheses have been processed.

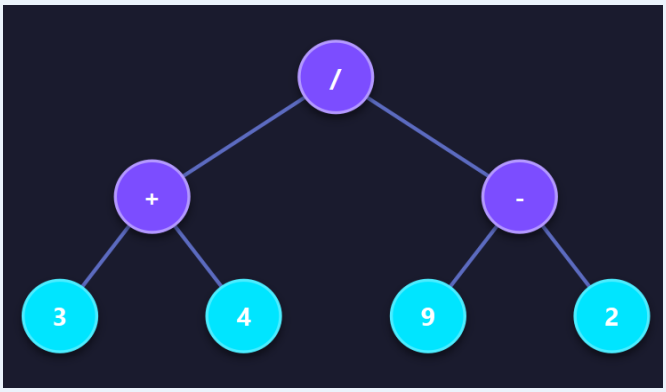
Operation	Description
push(x)	Place x on top of the stack
pop()	Remove and return the top item
peek()	Inspect the top item without removing it
is_empty()	Determine whether the stack contains items

3.2 Binary Expression Trees

An expression tree is a binary tree in which internal nodes represent operators and leaf nodes represent operands. Operands may be numeric values such as 3 and 4.5, or symbolic variables such as A, x, and z.

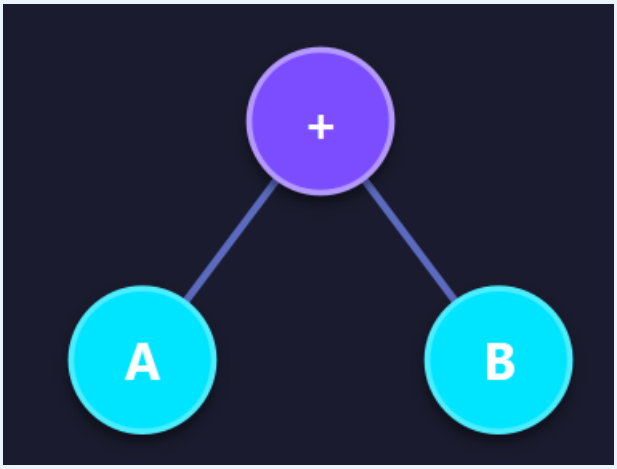
For the expression ((3+4)/(9-2)), the tree structure is:

Tree: ((3+4)/(9-2))



For the expression (A+B), the tree structure is:

Tree: (A+B)



Expression trees are valuable because a single structure yields multiple notation forms through traversal.

3.3 Traversals and Notation

Traversal	Visit Order	Produced Notation
Inorder	Left → Root → Right	Infix
Preorder	Root → Left → Right	Prefix
Postorder	Left → Right → Root	Postfix

For the expression (A+B), the three forms are:

Form	Output
Infix	A + B
Prefix	+ A B
Postfix	A B +

3.4 Operator Precedence

Precedence	Operators	Associativity
3 (Highest)	^	Right-to-left
2	*, /	Left-to-right
1 (Lowest)	+, -	Left-to-right

Parentheses override the default precedence order by explicitly grouping sub-expressions.

4 Tokenizer Description

The tokenizer in core/tokenizer.py performs lexical analysis by scanning the input string from left to right and producing typed tokens.

Each token stores:

Field	Meaning
value	The literal text for the token
token_type	One of NUMBER, VARIABLE, OPERATOR, LPAREN, RPAREN, or UNKNOWN
position	The zero-based source index used for diagnostics

The tokenizer recognizes:

- Multi-digit integers and decimal numbers.
- Single-letter symbolic variables from A–Z and a–z.
- Operators +, -, *, /, and ^.
- Parentheses (and).
- Unary minus when it prefixes a numeric literal.

Example token stream for (A+B):

Source	Token Type
`(`	LPAREN
`A`	VARIABLE
`+`	OPERATOR
`B`	VARIABLE
`)`	RPAREN

5 Algorithms Used

5.1 Infix to Postfix — Shunting-Yard Algorithm

The system uses the Shunting-Yard algorithm. Operands are sent directly to the output queue, while operators are temporarily stored on a stack until precedence and associativity rules allow them to be emitted.

Shunting-Yard Pseudocode

```
FOR each token T:
    IF T is an operand:
        append T to output
    ELSE IF T is an operator:
        pop higher-precedence operators from stack to output
        push T onto stack
    ELSE IF T is '(':
        push T onto stack
    ELSE IF T is ')':
        pop until '(' is encountered
```

5.2 Infix to Prefix

Prefix conversion is performed by reversing the infix token list, swapping parentheses, applying a precedence-aware conversion, and reversing the output sequence. This mirrors the postfix conversion logic while producing the prefix (Polish) form.

5.3 Expression Tree Construction

The expression tree is built from postfix notation using a stack of tree nodes.

Tree Construction Pseudocode

```
FOR each token T in postfix:
    IF T is an operand:
        push leaf node(T)
    ELSE IF T is an operator:
        right = pop()
        left  = pop()
        push node(T, left, right)
```

5.4 Evaluation

Postorder recursion evaluates a tree only when every leaf is numeric. If any leaf contains a symbolic variable, the tree remains available for visualization and traversal, while numeric evaluation is omitted. This allows the same infrastructure to serve both pedagogical and computational purposes.

6 Implementation Notes

6.1 Parser and Validation

The parser validates balanced parentheses, detects consecutive operators, rejects empty parentheses, and reports missing operators between adjacent operands or between an operand and an opening parenthesis. Operands are treated uniformly whether they are numeric literals or symbolic variables.

6.2 Tree Nodes

Each TreeNode stores a string value and optional left and right children. Leaf nodes represent operands, while internal nodes represent operators. This minimal design allows the same node type to model both operands (A, 3, x) and operators (+, /, *) uniformly.

6.3 Traversals

The traversal module implements recursive inorder, preorder, and postorder algorithms. The GUI uses the resulting node sequence to highlight the tree and show the corresponding notation output, enabling learners to follow the visitation path in real time.

6.4 Evaluation Behaviour

Numeric trees produce a calculation log and final result. Symbolic trees continue through tokenization, conversion, tree construction, and traversal without interruption, but the interface displays an evaluation notice instead of attempting arithmetic. This design ensures the tool is useful even when not all variables are bound to numeric values.

7 Sample Inputs

Expression	Category	Prefix	Postfix	Result
((3+4)/(9-2))	Numeric	/ + 3 4 - 9 2	3 4 + 9 2 - /	1
(A+B)	Symbolic	+ A B	A B +	N/A
x*(y+z)	Symbolic	* x + y z	x y z + *	N/A
((a+4)/(b-2))	Mixed	/ + a 4 - b 2	a 4 + b 2 - /	N/A
((x+y)/(z-w))	Symbolic	/ + x y - z w	x y + z w - /	N/A

8 System Architecture

The project follows a modular structure with a clear separation between algorithms, core data handling, interface widgets, and visualization support.

Module Dependency Tree

```
run.py
  └── app/main.py
        └── ui/main_window.py
              ├── ui/expression_input.py
              ├── ui/traversal_panel.py
              ├── ui/tree_canvas.py
              ├── ui/stack_visualizer.py
              ├── core/tokenizer.py
              ├── core/parser.py
              ├── core/infix_to_postfix.py
              ├── core/infix_to_prefix.py
              ├── core/expression_tree.py
              ├── core/traversal.py
              ├── core/evaluator.py
              └── algorithms/shunting_yard.py
```

The architecture enforces a one-way data flow: the UI layer dispatches expressions to the core modules, which return structured results that the visualization widgets render. No core module holds a reference to any UI widget, ensuring that the algorithmic logic can be tested independently.

9 Conclusion

The Expression Tree Visualizer demonstrates how expression parsing, notation conversion, tree construction, traversal, and evaluation interact within a single learning environment. By supporting both numeric literals and symbolic variables, the project provides a broader view of how expression trees represent structure independently of whether arithmetic evaluation is available.

The modular architecture keeps algorithmic concerns separate from the graphical interface, making it straightforward to extend the system with new operators, additional traversal orders, or alternative visualization modes. The application serves as both a functional tool and a pedagogical companion for students studying data structures and algorithms.